# FASE II

Manual de despliegue de Fase II
Asistente de preparación de revisiones de portafolio



*Vértice AI Consulting · Convertimos datos en decisiones*

| | |
|---|---|
| **Materia:** | *Gestión de Proyectos de Inteligencia Artificial* |
| **Actividad:** | Fase II · Módulo 2 / Semana 8 · Temas 15 y 16 |
| **Alumno(a):** | *Eduardo Aburto Angulo* |
| **Matrícula:** | *AL07182743* |
| **Docente:** | *M. en C. Luis Ariel Vázquez Piña* |
| **Fecha de entrega:** | 31 - 10 - 2026 |

# Declaración de Uso de Inteligencia Artificial

El alumno debe indicar de manera honesta si utilizó herramientas de Inteligencia Artificial (IA) para el desarrollo de esta actividad. Completa la siguiente tabla:

| Herramienta de IA utilizada | ¿La usaste? | Propósito / Descripción de uso |
|---|---|---|
| ChatGPT / OpenAI | Sí | Revisión 1.1.0 con ChatGPT/Codex: arquitectura, análisis retrospectivo, contrato OpenAPI y edición documental. No se reentrenó el modelo. Evidencias ejecutadas separadas de procedimientos preparados. |
| Claude (Anthropic) | No | No utilizado para esta fase. |
| Gemini (Google) | No | No utilizado para esta fase. |
| Copilot (Microsoft) | No | No utilizado para esta fase. |
| Otra (especifica): | No | No utilizado para esta fase. |

# Desarrollo de Fase II

Eduardo Aburto Angulo · 31 de octubre de 2026 · Versión 1.1.0

## Propósito y decisión

Integro el modelo académico de la Actividad 6 en una aplicación que puedo revisar como banquero Sr. Streamlit recoge la noticia o el escenario y FastAPI responde mediante HTTP. Docker proporciona el mismo entorno para ambos componentes. La ejecución local está comprobada; el procedimiento cloud siguiente está preparado y no se presenta como una ejecución realizada.

Mi prioridad es preservar el perfil del cliente y revisar su beneficio neto. El clasificador sólo detecta noticias negativas en inglés; no estima rentabilidad ni recomienda un fondo. Los 150 casos de superhéroes y villanos son ficticios, con posiciones de 5 a 20 millones MXN, para trabajar sin datos personales.

## Arquitectura e integración

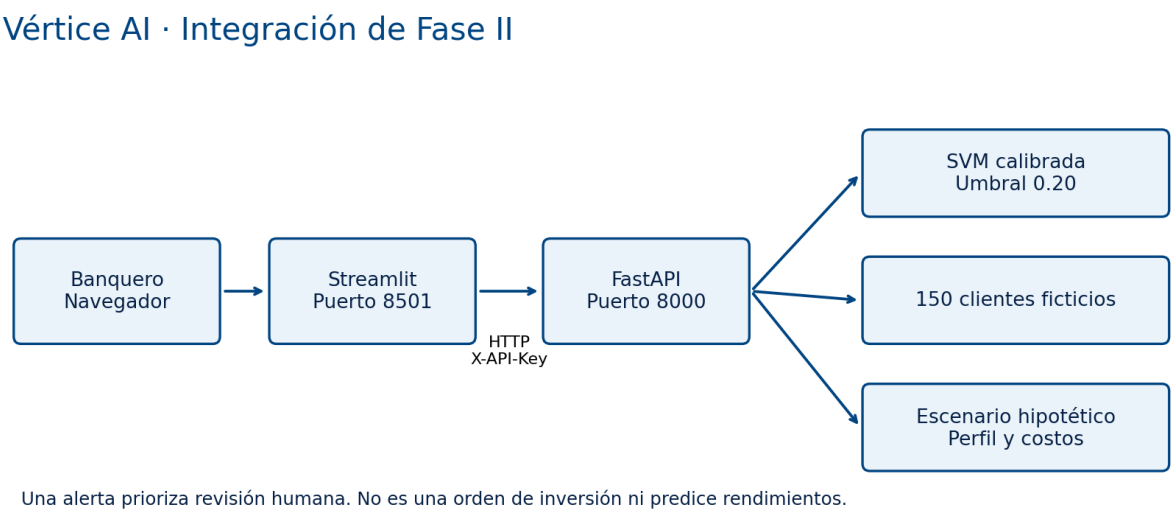



Figura 1. Arquitectura de la solución: navegador, Streamlit, FastAPI y modelo conservado.

El navegador conversa con Streamlit; sólo el servidor Streamlit conoce la clave y llama a FastAPI por la red de Compose. Al iniciar la API, antes de aceptar peticiones, se verifica el SHA256 del modelo contra el manifiesto. Esa comprobación ocurre al arranque, no en cada noticia. El archivo conservado se carga únicamente si coincide.

Secuencia de una revisión: (1) el banquero escribe una noticia en inglés; (2) Streamlit envía POST a http://api:8000/v1/predict con X-API-Key; (3) FastAPI valida el esquema, la credencial y la presencia de vocabulario conocido; (4) el modelo ya verificado produce la probabilidad; (5) la API compara con 0.20; (6) la pantalla muestra probabilidad, prioridad y la leyenda de revisión humana. Un error de red se presenta de forma comprensible y no se sustituye por una predicción inventada.

| Ruta y método | Autenticación | Entrada | Salida | Errores previstos |
|---|---|---|---|---|
| GET /health | Pública | Ninguna | Estado y disponibilidad del modelo | 503 |
| GET /v1/model | X-API-Key | Ninguna | Manifiesto, hash y limitaciones | 401 |

| Ruta y método | Autenticación | Entrada | Salida | Errores previstos |
|---|---|---|---|---|
| POST /v1/predict | X-API-Key | texto; idioma en | Probabilidad, umbral y aviso | 401, 422, 503 |
| GET /v1/clients | X-API-Key | Ninguna | 150 clientes ficticios | 401 |
| POST /v1/scenario | X-API-Key | Cliente, proporción, mejora, meses, gastos, perfil | Compatibilidad y beneficio supuesto | 401, 404, 422 |

La ruta de salud es pública; las cuatro rutas de negocio requieren clave. El contrato completo de campos y restricciones está en documentos/openapi.json. Los errores no convierten la API en un sistema de autorización de operaciones.

## Peticiones y respuestas observadas

Ejecuté los cinco casos con FastAPI TestClient y su ciclo de arranque real, sin alterar la lógica. evidencias/ejemplos_api.json conserva las respuestas completas. Los siguientes extractos seleccionan campos de esas respuestas; no se presentan como cuerpos completos. En una llamada HTTP real se añade X-API-Key desde el entorno, sin publicar su valor.

GET /health — HTTP 200.


```
{
  "entrada": null,
  "salida_extracto": {
    "status": "ok",
    "model_ready": true,
    "version": "1.0.0"
  }
}
```


GET /v1/model — HTTP 200.


```
{
  "entrada": null,
  "salida_extracto": {
    "sha256": "f48c7496d5a12bd15d535721e3e83c6ef895b1ae7eaccbe7f4f4c60705bb1916",
    "notice": "Priorizar revisión humana; no es orden de inversión ni predicción de
rendimiento.",
    "joint_target_met": false
  }
}
```


POST /v1/predict — HTTP 200.


```
{
  "entrada": {
    "texto": "The company reported a decline in operating profit.",
    "idioma": "en"
  },
  "salida_extracto": {
    "probabilidad_negativa": 0.3544057081827505,
    "umbral": 0.2,
    "priorizar_revision": true,
    "idioma_declarado": "en",
    "notice": "Priorizar revisión humana; no es orden de inversión ni predicción de
rendimiento.",
    "model_sha256": "f48c7496d5a12bd15d535721e3e83c6ef895b1ae7eaccbe7f4f4c60705bb1916"
  }
}
```


GET /v1/clients — HTTP 200.


```
{
  "entrada": null,
  "salida_extracto": {
    "simulados": true,
```

```
    "clientes": [
      {
        "cliente_id": "C001",
        "nombre": "Killer Croc",
        "arquetipo": "Villano",
        "perfil_riesgo": "Moderado",
        "posicion_mxn": 11858000,
        "fondo_actual": "BBVACR",
        "serie_actual": "P",
        "categoria_actual": "Fondos de Estrategias Pesos",
        "rend_1_anio_actual": 11.93,
        "clase_activo": "Deuda Corto Plazo",
        "meses_sin_revision": 6
      }
    ]
  }
}
```


POST /v1/scenario — HTTP 200.


```
{
  "entrada": {
    "cliente_id": "C001",
    "proporcion": 0.35,
    "mejora_anual_pp": 3,
    "meses": 12,
    "gastos_adicionales_anuales_pp": 0,
    "perfil_destino": "Moderado"
  },
  "salida_extracto": {
    "simulado": true,
    "perfil_compatible": true,
    "monto_migrado_mxn": 4150299.9999999995,
    "mejora_supuesta_mxn": 124508.99999999999,
    "corretaje_supuesto_mxn": 83005.99999999999,
    "gastos_adicionales_mxn": 0.0,
    "beneficio_incremental_supuesto_mxn": 41503.0,
    "elegible_para_revision": true,
    "supuesto": "1 % compra + 1 % venta sobre el mismo nocional. Cálculo lineal, sin impuestos ni
garantía. Gastos adicionales sólo si no están ya descontados del rendimiento. No equivale al
ingreso neto del banco.",
    "notice": "Priorizar revisión humana; no es orden de inversión ni predicción de rendimiento."
  }
}
```


## Estrategia de despliegue: tres rutas evaluadas

Comparo el control operativo, la facilidad de acceso del docente y la superficie expuesta antes de elegir infraestructura. Los importes son referencias de catálogo en USD consultadas el 30 de octubre de 2026, no cargos observados ni presupuestos aprobados. El cálculo reproducible y los supuestos están en evidencias/comparacion_costos.json (Google Cloud, s. f.-a; Streamlit, s. f.; Render, s. f.).

| Ruta | Esfuerzo de puesta en marcha | Costo mensual aproximado | Portabilidad | Exposición de red | Cuándo la elegiría |
|---|---|---|---|---|---|
| IaaS Compute Engine y túnel SSH | Mayor: administrar Ubuntu, Docker, actualizaciones y respaldo | USD 24.46 sólo por e2-medium durante 730 horas; disco, IP y transferencia aparte | Alta con Docker; el sistema operativo queda a mi cargo | SSH restringido; aplicación accesible por túnel | Demostración privada con control de la máquina |

| Ruta | Esfuerzo de puesta en marcha | Costo mensual aproximado | Portabilidad | Exposición de red | Cuándo la elegiría |
|---|---|---|---|---|---|
| PaaS de contenedores Cloud Run | Intermedio: imagen, IAM, secretos y parámetros de servicio | USD 0 de cómputo si el uso queda en la cuota gratuita; almacenamiento de imagen, secretos y red se evalúan aparte; excedentes por consumo | Alta: contenedor OCI; IAM y configuración son específicos de Google | HTTPS gestionado; web pública sólo para demostración, API protegida por IAM | Piloto con servicios separados y operación administrada |
| PaaS de aplicación Streamlit Community Cloud | Menor para la pantalla; requiere adaptar el arranque del backend o alojarlo aparte | USD 0 de alojamiento Community Cloud, sujeto a límites del servicio | Media: despliega Python desde GitHub; no ejecuta nuestro Compose | URL HTTPS de la aplicación; no publica automáticamente FastAPI | Acceso rápido del docente a una demostración académica |
| Alternativa gratuita Render | Intermedio: dos servicios Docker, variables y clave compartida | USD 0 en Free dentro de las cuotas; recursos limitados y suspensión por inactividad | Alta para esta imagen; configuración propia de Render | Dos URLs HTTPS; endpoints API protegidos con X-API-Key | Demostración pública sin administrar una VM |

El supuesto de 730 horas sirve únicamente para comparar un mes de uso continuo. La tarifa e2-medium consultada es USD 0.03350571 por hora en Iowa; 730 × 0.03350571 = USD 24.4591683. No es el costo total de la VM ni una medición de gasto. En Cloud Run, escala a cero y HTTPS gestionado no significan costo total garantizado de cero; las conexiones WebSocket de Streamlit pueden mantener una instancia activa (Google Cloud, s. f.-a, s. f.-b).

Aclaro una limitación de arquitectura: frontend y backend no tienen que convivir en un mismo proceso en estos PaaS. Community Cloud administra el proceso Streamlit y no ofrece Compose como interfaz de despliegue; puedo iniciar un backend local bajo un supervisor, con ciclo de vida acoplado, o mantenerlo como servicio externo. Hugging Face Docker Spaces también permite varios procesos en un contenedor, pero eso no equivale a dos servicios independientes. No describo como gratuita una cuenta nueva de cómputo en Spaces: la documentación vigente exige un plan de pago para crear Spaces Docker o Gradio aunque CPU Basic no tenga precio por hora (Hugging Face, s. f.).

Elijo conservar Compute Engine con túnel como referencia de demostración privada y mantener Render Free como alternativa opcional, no ejecutada ni necesaria para esta rúbrica. Para un piloto posterior elegiría Cloud Run con autenticación y gobierno de secretos. Con ánimo de avanzar sin aumentar el riesgo reputacional, separo una URL demostrativa de una autorización para utilizar datos u operaciones reales.

## Requisitos y componentes

| Requerimiento técnico | Configuración y alcance |
|---|---|
| Sistema operativo | Mac o Windows con Docker Desktop; Linux Ubuntu 24.04 con systemd para el instalador de Engine |
| CPU | ARM64 o AMD64; 2 vCPU iniciales propuestas; sin GPU |
| RAM y disco | 4 GB RAM y 20 GB de disco iniciales propuestos; no constituyen prueba de capacidad |

| Requerimiento técnico | Configuración y alcance |
|---|---|
| Python | 3.12 dentro de la imagen; Python 3.12 y venv para pruebas fuera de ella |
| Docker y Compose | Engine con BuildKit y Compose v2 compatible con --wait; versiones ejecutadas en evidencias/docker-version.txt y evidencias/replica_05_versiones.txt |
| Puertos locales | 127.0.0.1:8000 API y 127.0.0.1:8501 pantalla; deben estar libres |
| Variables | API_TOKEN de al menos 16 caracteres; SERVICE api/web; API_URL http://api:8000 para web; PORT según servicio |
| Salida de red | GitHub para código, Docker Hub para imagen base, PyPI para dependencias; apt de Ubuntu y Docker al instalar Engine; CDN de Swagger al consultar documentación |

Necesito también Python 3 para generar .env. En Ubuntu el instalador de Engine requiere systemd activo: un contenedor ordinario Ubuntu no arranca ese servicio. Para pruebas en contenedor uso el Docker ya operativo del host o una VM adecuada; no afirmo que instalar paquetes equivalga a iniciar el daemon. Necesito Git, Docker Desktop abierto en Mac/Windows o Docker Engine con Compose en Linux. La referencia cloud es Ubuntu 24.04 LTS, 2 vCPU, 4 GB RAM y 20 GB de disco; son recursos iniciales propuestos, no una garantía de capacidad ni una cotización. No se necesita GPU. La construcción requiere salida a Docker Hub y al repositorio de paquetes Python.

El Dockerfile utiliza Python 3.12 y dependencias directas fijadas. El proceso corre como usuario 10001. Compose monta un filesystem de sólo lectura, permite temporales en /tmp y retira capacidades Linux. La API comprueba el SHA256 del modelo antes de cargarlo. SERVICE selecciona api o web; PORT permite adaptar el puerto a otros servicios de contenedores.

El navegador accede a Streamlit en 8501. Streamlit consulta http://api:8000 mediante la red de Compose y un encabezado X-API-Key. La API proporciona /health, /v1/model, /v1/predict, /v1/clients y /v1/scenario. Los puertos del host sólo escuchan en 127.0.0.1. El usuario no necesita conocer la clave para usar la pantalla local; por eso el acceso a la pantalla debe permanecer privado.

## Ejecución local paso a paso

Desde una terminal, clono el repositorio y genero una clave aleatoria. Si ya existe .env, conservo su contenido o hago una copia antes de reemplazarlo. No subo ese archivo a GitHub.

```
git clone https://github.com/eduardoaburtoa/fase2-despliegue-portafolios.git
cd fase2-despliegue-portafolios
python3 -c "import secrets,pathlib;
pathlib.Path('.env').write_text('API_TOKEN='+secrets.token_urlsafe(32)+'\n')"
chmod 600 .env
docker compose config --quiet
docker compose up --build -d --wait --wait-timeout 180
docker compose ps
```

Espero que api y web aparezcan healthy. Abro http://localhost:8501, pulso Preparar revisión y compruebo la probabilidad 35.4 % del ejemplo. En Escenario del cliente elijo un perfil compatible y comparo los costos; un perfil distinto debe bloquear la elegibilidad. La documentación interactiva está en http://localhost:8000/docs; sus endpoints protegidos necesitan X-API-Key.

Para comprobar la red entre contenedores sin copiar credenciales:

```
docker compose exec -T web python -c "from frontend.client import call; print(call('/v1/model')
['sha256'])"
docker compose logs --tail 50
```

Para ejecutar la suite completa uso Python 3.12 en un entorno separado. Las instrucciones reproducibles están en README. La regresión HTTP recorre 452 noticias y guarda métricas; no reentrena ni selecciona otro umbral. Al terminar apago los servicios con docker compose down. Esto no borra los archivos fuente.

### Validación completa desde la carpeta del proyecto

Con los servicios encendidos, ejecuto los pasos siguientes. Python 3.12 con venv debe estar instalado; en Ubuntu instalo python3-venv. Las dependencias de documentación sólo son necesarias para regenerar el histograma.

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-docs.txt
python -m pytest pruebas -q --junitxml=evidencias/pytest-local.xml
set -a
source .env
set +a
TEST_ENV=docker_local python scripts/smoke.py
TEST_ENV=docker_local python scripts/regression_http.py
python scripts/export_docs.py
python scripts/analyze_behavior.py
docker compose down
```

## Preparación de Google Cloud

Propongo una máquina Compute Engine y un túnel SSH para una demostración privada. Antes de crear recursos reviso facturación, presupuesto y permisos de mi proyecto. Esta guía no implica que se haya contratado ni desplegado un servidor. Necesito la CLI gcloud autenticada y permisos para habilitar Compute, crear red, firewall y máquina. No asigno al contenedor permisos sobre servicios cloud.

Defino variables propias en mi equipo; sustituyo MI_PROYECTO y MI_IP_PUBLICA por valores reales. La IP permitida es la del equipo autorizado, no 0.0.0.0/0. Uso nombres nuevos para evitar modificar infraestructura existente.

```
export FASE2_PROJECT='MI_PROYECTO'
export FASE2_REGION='us-central1'
export FASE2_ZONE='us-central1-a'
export FASE2_CIDR='MI_IP_PUBLICA/32'
gcloud config set project "$FASE2_PROJECT"
gcloud services enable compute.googleapis.com
gcloud compute networks create vertice-fase2 --subnet-mode=custom
gcloud compute networks subnets create vertice-fase2-subnet \
  --network=vertice-fase2 --region="$FASE2_REGION" --range=10.20.0.0/24
gcloud compute firewall-rules create vertice-fase2-ssh \
  --network=vertice-fase2 --allow=tcp:22 \
  --source-ranges="$FASE2_CIDR" --target-tags=vertice-fase2-ssh
gcloud compute instances create vertice-fase2 \
  --zone="$FASE2_ZONE" --machine-type=e2-medium \
  --subnet=vertice-fase2-subnet --tags=vertice-fase2-ssh \
  --image-family=ubuntu-2404-lts-amd64 --image-project=ubuntu-os-cloud \
  --boot-disk-size=20GB --no-service-account --no-scopes
gcloud compute ssh vertice-fase2 --zone="$FASE2_ZONE"
```

No abro 8000 ni 8501 en el firewall cloud. Si una política corporativa impide IP externa o SSH directo, uso el mecanismo de acceso aprobado por el administrador, por ejemplo IAP, y adapto la red antes de continuar.

# Instalación en la máquina y acceso privado

Dentro de la máquina Ubuntu instalo Docker siguiendo el repositorio apt oficial. El script scripts/install_docker_ubuntu.sh documenta las mismas operaciones; lo reviso y lo ejecuto únicamente en Ubuntu 24.04 preparado para esta demostración. No fue ejecutado en un servidor cloud durante esta entrega.

```
sudo apt-get update
sudo apt-get install -y git python3 ca-certificates curl
git clone https://github.com/eduardoaburtoa/fase2-despliegue-portafolios.git
cd fase2-despliegue-portafolios
sudo bash scripts/install_docker_ubuntu.sh
python3 -c "import secrets,pathlib;
pathlib.Path('.env').write_text('API_TOKEN='+secrets.token_urlsafe(32)+'\n')"
chmod 600 .env
sudo docker compose up --build -d --wait --wait-timeout 180
sudo docker compose ps
sudo docker compose exec -T web python -c "from frontend.client import call;
print(call('/v1/model')['sha256'])"
```

En otra terminal de mi equipo abro el túnel. Si Docker local sigue usando los puertos, lo detengo primero o uso puertos locales alternativos 18501 y 18000.

```
gcloud compute ssh vertice-fase2 --zone="$FASE2_ZONE" -- \
  -N -L 18501:127.0.0.1:8501 -L 18000:127.0.0.1:8000
```

Abro http://localhost:18501 y http://localhost:18000/docs. Repito noticia, escenario, credencial inválida y salud. Guardo la revisión Git, ID de imagen, arquitectura y fecha, junto con capturas. Una construcción ARM64 local no acredita por sí sola una ejecución AMD64 cloud; ese paso debe comprobarse en la máquina de destino.

# Operación y reversión

Registro git rev-parse HEAD y docker image inspect vertice-fase2:1.0.0. Antes de una actualización guardo la imagen validada con docker tag vertice-fase2:1.0.0 vertice-fase2:respaldo y conservo el commit anterior. Si falla la nueva versión, recupero el commit anterior, restauro la etiqueta desde respaldo y uso docker compose up -d --no-build --pull never. No elimino el respaldo hasta aceptar la actualización.

Reviso salud, errores y consumo con docker compose ps, logs y docker stats. No registro textos completos, claves ni datos personales. Para rotar la clave actualizo .env y recreo ambos servicios con docker compose up -d --force-recreate. Antes de un piloto real faltan autenticación individual, TLS para exposición web, límites de solicitudes, gestión de secretos, pruebas de carga y validación externa con datos aptos para ese uso.

Si falla la construcción, reviso conexión y espacio; si falta el token, reviso .env; si la API no está healthy, consulto sus logs e integridad del modelo. Un 401 indica clave inválida y un 422 datos no válidos o texto sin vocabulario conocido. No resuelvo un error de integridad omitiendo el control.

Detener una VM no elimina cargos de disco. Tras la demostración decido si conservo o elimino los recursos desde Google Cloud, verificando antes qué información se perdería. No se ha ejecutado esa eliminación como parte de esta entrega.

# Fuentes y asistencia documental

Docker: https://docs.docker.com/compose/how-tos/startup-order/ y https://docs.docker.com/engine/install/ubuntu/

Google Cloud: https://docs.cloud.google.com/compute/docs/instances/create-start-instance y https://docs.cloud.google.com/compute/docs/connect/standard-ssh

El código conserva el modelo de https://github.com/eduardoaburtoa/actividad6-evaluacion-portafolios . La documentación y las pruebas se elaboraron con apoyo de ChatGPT/Codex; no atribuyo a Copilot un uso que no realicé. Distingo la ejecución local verificada del procedimiento cloud preparado.

## Continuidad del proyecto

Mi proyecto parte del motor de señales de rotación de fondos de inversión con IA para banca patrimonial, privada y casa de bolsa. La Actividad 4 comparó modelos preentrenados de Hugging Face para leer notas de mercado; la Actividad 5 registró el entrenamiento en MLflow; la Actividad 6 evaluó la SVM calibrada con umbral 0.20. La Fase I construyó el business case y esta Fase II convierte el modelo conservado en una aplicación integrada y desplegable.

Conservo las cifras de mi business case sin recalcularlas: alcance de 24 asesores por 150 clientes, 3,600 clientes y AUM simulado de $54,397 millones; inversión del piloto de $9,083,448 en los meses 1 a 9 con seis asesores; operación anual de $4,769,996. El beneficio esperado a pleno funcionamiento es $40,540,443 al año, con ROI a tres años de 298.6 % y recuperación en el mes 13; el escenario conservador plantea 38.8 % y mes 25 (Aburto Angulo, 2026).

La razón económica del proyecto es que, bajo aquellos supuestos, los clientes pagarían $79.65 millones de corretaje y conservarían $82.72 millones de rendimiento adicional: $1.04 por cada peso de corretaje. El umbral de rotación planteado es una mejora de 3.0 puntos porcentuales frente al costo supuesto de 2 %. Los supuestos críticos son S-14, detección actual de 30 %; S-15, aceptación de 70 %; y S-16, seis rotaciones no justificadas al año. Son hipótesis del caso de negocio, no resultados de esta aplicación.

Materializo la lectura de noticias con PLN y un escenario que calcula el costo didáctico del 2 % y la compatibilidad del perfil antes de proponer una revisión. Quedan para Fase III la proyección de rendimiento por clase de activo, la combinación de señales con el umbral financiero de 3.0 puntos, la integración con órdenes y el tablero de KPI. El umbral financiero de 3.0 puntos no debe confundirse con la probabilidad 0.20 del clasificador.

El riesgo R-01 es el conflicto de interés: la institución gana cuando el cliente se mueve. El control previsto exige suspender y recalibrar el motor si más de 25 % de las rotaciones recomendadas no supera su costo a doce meses. Este control todavía no está automatizado. La Fase II acredita que la arquitectura funciona, no que el ROI de 298.6 % se vaya a materializar; eso se mide en el piloto de nueve meses. La IA no sustituye el criterio del banquero; lo aumenta.

## Herramientas de documentación y contrato reproducible

FastAPI genera /openapi.json desde las rutas y los esquemas Pydantic (FastAPI, s. f.). Ejecuté scripts/export_docs.py para congelar ese contrato y obtener cinco respuestas reales mediante TestClient. La referencia documentos/api-reference.html permite navegarlo con Swagger UI; se abre descargando el HTML y requiere conexión al CDN. GitHub muestra su código y no lo ejecuta como una aplicación web. La copia estática se actualiza al volver a ejecutar el script; no se actualiza sola después de un cambio.

```
python scripts/export_docs.py
```

No atribuyo a Copilot un uso que no realicé. Utilicé ChatGPT/Codex para asistencia de programación y edición, FastAPI para OpenAPI y Swagger UI para la referencia navegable. La revisión de estas salidas forma parte de la integración y documentación de los temas 15 y 16 (Universidad Tecmilenio, 2026a, 2026b).

## Replicabilidad y alcance de la comprobación

La repetición completó las 23 pruebas en Ubuntu limpio. Después de registrar el fallo por ausencia de systemd, continué en un contenedor de laboratorio con daemon Docker propio y almacenamiento vfs, sin montar el socket ni carpetas del host. Ese contenedor externo necesita privilegios sólo para Docker anidado; no cambia el usuario ni los controles de los servicios de la aplicación. Se construyó la imagen, ambos servicios alcanzaron healthy, las 452 llamadas reprodujeron las predicciones y el cliente Streamlit consultó la API por nombre de servicio. Este modo es una comprobación de laboratorio y no una recomendación de despliegue productivo. Las bitácoras y los tiempos están en evidencias/replicabilidad.json; la regresión está en evidencias/regresion_ubuntu_limpio.json.

## Conclusiones

Con ánimo de dejar una entrega que otro banquero pueda revisar, documenté la arquitectura, el contrato y los prerrequisitos de cada ruta. Mi elección para esta etapa es la demostración local privada y una guía de nube preparada. Docker y sus controles de arranque facilitan consistencia, pero no sustituyen el gobierno de acceso ni el análisis del cliente (Docker, Inc., s. f.-a, s. f.-b). Una eventual pantalla alojada debe respetar las condiciones de Streamlit (s. f.) y el modelo de identidad del proveedor (Google Cloud, s. f.-c, s. f.-d). La comisión didáctica se distingue de las condiciones por fondo y serie publicadas por BBVA México (s. f.).

## Procedimiento adicional para Cloud Run

Procedimiento preparado; requiere proyecto con facturación habilitada. No ejecutado en Google Cloud en esta revisión. Usa la misma API y una imagen web derivada para añadir identidad IAM sin modificar el frontend original.

```
export FASE2_PROJECT='MI_PROYECTO'
export FASE2_REGION='us-central1'
export FASE2_REG="$FASE2_REGION-docker.pkg.dev/$FASE2_PROJECT/vertice"
export FASE2_BASE="$FASE2_REG/api:1.1.0"
export FASE2_WEB="$FASE2_REG/web:1.1.0"
gcloud config set project "$FASE2_PROJECT"
gcloud services enable run.googleapis.com artifactregistry.googleapis.com
secretmanager.googleapis.com
gcloud artifacts repositories create vertice --repository-format=docker --
location="$FASE2_REGION"
gcloud auth configure-docker "$FASE2_REGION-docker.pkg.dev"
docker buildx build --platform linux/amd64 -t "$FASE2_BASE" --push .
docker buildx build --platform linux/amd64 -f deploy/cloudrun.Dockerfile \
  --build-arg BASE_IMAGE="$FASE2_BASE" -t "$FASE2_WEB" --push .
gcloud iam service-accounts create vertice-api
gcloud iam service-accounts create vertice-web
python3 -c "import secrets; print(secrets.token_urlsafe(32), end='')" | \
  gcloud secrets create vertice-api-token --data-file=-
for FASE2_SA in vertice-api vertice-web; do
  gcloud secrets add-iam-policy-binding vertice-api-token \
    --member="serviceAccount:$FASE2_SA@$FASE2_PROJECT.iam.gserviceaccount.com" \
    --role=roles/secretmanager.secretAccessor
done
gcloud run deploy vertice-api --image="$FASE2_BASE" \
  --region="$FASE2_REGION" --service-account="vertice-api@$FASE2_PROJECT.iam.gserviceaccount.com"
\
  --no-allow-unauthenticated --port=8000 --set-env-vars=SERVICE=api \
  --set-secrets=API_TOKEN=vertice-api-token:1 --min-instances=0 --max-instances=1 \
  --memory=1Gi --cpu=1
export FASE2_API_URL="$(gcloud run services describe vertice-api --region="$FASE2_REGION" --
format='value(status.url)')"
```

```
gcloud run services add-iam-policy-binding vertice-api --region="$FASE2_REGION" \
  --member="serviceAccount:vertice-web@$FASE2_PROJECT.iam.gserviceaccount.com"
--role=roles/run.invoker
gcloud run deploy vertice-web --image="$FASE2_WEB" \
  --region="$FASE2_REGION" --service-account="vertice-web@$FASE2_PROJECT.iam.gserviceaccount.com"
\
  --allow-unauthenticated --port=8080 --set-env-vars="CLOUD_RUN_API_URL=$FASE2_API_URL" \
  --set-secrets=API_TOKEN=vertice-api-token:1 --min-instances=0 --max-instances=1 \
  --memory=1Gi --cpu=1 --timeout=3600
```

Los nombres del ejemplo deben estar libres; si ya existen, revisar los recursos en vez de recrearlos. Cloud Run inyecta PORT según --port; el arranque de la API y del adaptador web lo respetan. El secreto se fija a la versión 1; una rotación exige desplegar ambos servicios con la nueva versión.

`--allow-unauthenticated` sólo se utiliza en web para que el docente pueda abrir la demostración en su navegador sin credenciales de Google Cloud. No es una obligación de Streamlit ni de Cloud Run: para un piloto privado se retira y se añade acceso autenticado. La API conserva IAM privado y X-API-Key. La pantalla pública permite usar el modelo a través de web; por tanto sólo se despliegan datos ficticios y no es una frontera de autorización bancaria.

El proxy local obtiene un ID token de la identidad vertice-web para la audiencia de la API. Una API key sola no satisface IAM. No se pone el secreto en Dockerfile, GitHub ni el navegador. La demostración pública no utiliza clientes reales y no ejecuta operaciones.

Para probar la API privada se necesita además un ID token autorizado en el cliente de prueba. scripts/smoke.py se conserva sin cambios y no implementa IAM: se usa contra la API pública de Render protegida con X-API-Key, o contra localhost. En Cloud Run se debe usar un cliente con Authorization Bearer y X-API-Key, además de comprobar la pantalla y los logs.

## Cobertura textual de la rúbrica

Los cuatro criterios tienen ponderación de 25 % (Universidad Tecmilenio, 2026c). Las páginas corresponden al PDF de esta revisión; Google Docs puede repaginar según su configuración.

| Criterio | Sección verificable | Página del PDF |
|---|---|---|
| Contenerización y despliegue local | Requisitos y componentes; Ejecución local paso a paso | 6–8 |
| Integración técnica (API, frontend/backend) | Arquitectura e integración; contrato y ejemplos | 3–5 |
| Pruebas y validación del modelo | Replicabilidad y alcance; documento complementario Validación y pruebas | 11; validación 6–9 |
| Documentación y repositorio | Herramientas de documentación; manual y rutas evaluadas | 5–13 |

## Referencias

Aburto Angulo, E. (2026). Fase I Business case del motor de señales de rotación de fondos de inversión con IA [Trabajo académico, Universidad Tecmilenio]. Cifras conservadas del caso de negocio y de las instrucciones de continuidad del autor.

BBVA México. (s. f.). Normativa de fondos. https://www.bbva.mx/personas/productos/inversion/normativa-de-fondos.html

Docker, Inc. (s. f.-a). Control startup and shutdown order in Compose. https://docs.docker.com/compose/how-tos/startup-order/

Docker, Inc. (s. f.-b). Install Docker Engine on Ubuntu. https://docs.docker.com/engine/install/ubuntu/

FastAPI. (s. f.). Extending OpenAPI. https://fastapi.tiangolo.com/how-to/extending-openapi/

Google Cloud. (s. f.-a). General-purpose machine family pricing. https://cloud.google.com/products/compute/pricing/general-purpose

Google Cloud. (s. f.-b). Cloud Run pricing. https://cloud.google.com/run/pricing

Google Cloud. (s. f.-c). Authenticating service-to-service. https://docs.cloud.google.com/run/docs/authenticating/service-to-service

Google Cloud. (s. f.-d). Configure secrets for services. https://docs.cloud.google.com/run/docs/configuring/services/secrets

Hugging Face. (s. f.). Spaces overview. https://huggingface.co/docs/hub/en/spaces-overview

Malo, P., Sinha, A., Korhonen, P., Wallenius, J., & Takala, P. (2014). Good debt or bad debt: Detecting semantic orientations in economic texts. Journal of the Association for Information Science and Technology, 65(4), 782–796. https://doi.org/10.1002/asi.23062

Render. (s. f.). Deploy for free. https://render.com/docs/free

Streamlit. (s. f.). Streamlit Community Cloud. https://docs.streamlit.io/deploy/streamlit-community-cloud

Universidad Tecmilenio. (2026a). Tema 15 [Material del curso Gestión de Proyectos de Inteligencia Artificial].

Universidad Tecmilenio. (2026b). Tema 16 [Material del curso Gestión de Proyectos de Inteligencia Artificial].

Universidad Tecmilenio. (2026c). Rúbrica del Proyecto Fase II [Instrumento de evaluación].

Este documento fue estructurado precisamente para cubrir esos 4 apartados de forma explícita y verificable.